

Livro de Programação Competitiva — OBI

Módulo 2 — Busca (DFS, BFS, 0-1 BFS, Dijkstra)

Este é o módulo que ataca diretamente sua dificuldade no simulado OBI 2015 — grafo/matriz e caminho pela matriz. Leia com calma na primeira vez; depois, use como referência de 30 minutos.

2.A — DFS (Busca em Profundidade)

1. Introdução

Objetivo: reconhecer quando usar DFS, implementar recursivo e iterativo, e aplicar em flood fill / componentes conexas.

Pré-requisitos: recursão, pilha, representação de grafos (lista de adjacência e grid implícito).

Quando estudar: antes de BFS (DFS costuma ser mais intuitivo primeiro, embora BFS seja mais usado em "menor caminho").

Nível: ★ Fácil (DFS simples) → ★ ★ Médio (flood fill em grid, componentes conexas)

2. Intuição

Imagine um labirinto onde você anda sempre em frente até bater numa parede, aí volta e tenta outro caminho — como o método de "mão na parede" para sair de um labirinto. Você vai **fundo** antes de voltar. Isso é DFS: aprofunda o máximo possível antes de retroceder (*backtrack*).

3. Problema que motivou

Você tem um mapa de terra e água (grid de 0s e 1s) e precisa contar quantas ilhas (grupos de terra conectada) existem. Para cada célula de terra ainda não visitada, você quer "pintar" toda a ilha conectada a ela de uma vez — isso é *flood fill*, feito com DFS (ou BFS).

4. Construindo a solução

Solução 1 (força bruta): para cada célula de terra, verificar manualmente se já foi contada, comparando com todas as anteriores → lento e complicado.

Solução final: para cada célula de terra não visitada, faça DFS marcando toda a ilha como visitada, e incremente o contador de ilhas uma vez por chamada → $O(N \cdot M)$, cada célula visitada uma única vez.

5. Estrutura do algoritmo

```
dfs(nó):  
    marca nó como visitado  
    para cada vizinho do nó:  
        se vizinho não visitado:  
            dfs(vizinho)
```

6. Implementação manual — grafo com lista de adjacência (recursiva)

```
cpp  
  
vector<vector<int>> adj;  
vector<bool> visited;  
  
void dfs(int u) {  
    visited[u] = true;  
    // processar u aqui, se necessário (ex: contar tamanho da componente)  
    for (int v : adj[u]) {  
        if (!visited[v]) {  
            dfs(v);  
        }  
    }  
}
```

7. Implementação STL — grid (matriz), o seu ponto de atenção

```
cpp
```

```

int n, m;
vector<vector<int>> grid;           // 0 = terra, 1 = parede, por exemplo
vector<vector<bool>> visited;

int dx[] = {-1, 1, 0, 0};
int dy[] = {0, 0, -1, 1};

void dfs(int i, int j) {
    visited[i][j] = true;
    for (int dir = 0; dir < 4; dir++) {
        int ni = i + dx[dir], nj = j + dy[dir];
        if (ni < 0 || ni >= n || nj < 0 || nj >= m) continue; // limites
        if (grid[ni][nj] == 1) continue;                     // obstáculo
        if (visited[ni][nj]) continue;                       // já visitado
        dfs(ni, nj);
    }
}

```

DFS iterativo (com pilha explícita — evita estouro em grafos grandes)

```

cpp

stack<int> st;
st.push(origem);
visited[origem] = true;

while (!st.empty()) {
    int u = st.top(); st.pop();
    for (int v : adj[u]) {
        if (!visited[v]) {
            visited[v] = true;
            st.push(v);
        }
    }
}

```

8. Complexidade

Operação	Complexidade
DFS completo (grafo com V nós, E arestas)	$O(V + E)$
DFS completo (grid $n \times m$)	$O(n \cdot m)$

```

Grid:          Ordem de visita (DFS a partir de (0,0)):
1 1 0          1 → 2 → 3
1 0 0          desce, desce, direita...
0 0 1          (vai fundo antes de "espalhar")

```

10. Erros clássicos

- Esquecer de marcar `visited` **antes** de continuar a recursão → loop infinito
- Não checar limites antes de acessar `grid[ni][nj]` → acesso inválido
- Usar DFS recursivo em grid muito grande (ex: 1000×1000) sem cuidado → risco de estouro de pilha; considere DFS iterativo ou BFS
- Achar que DFS encontra **menor** caminho — DFS não garante isso, apenas conectividade/alcançabilidade

11. Problemas clássicos

1. Contar componentes conexas num grafo simples (fácil)
2. Contar ilhas num grid (flood fill) (médio)
3. Detectar ciclo em grafo direcionado usando DFS com 3 estados (difícil)

12. Padrões — quando pensar em DFS

Se o enunciado diz...	Pense em...
Contar "grupos"/"regiões"/"ilhas" conectadas	DFS (flood fill)
Detectar ciclo	DFS
Verificar se existe caminho (sem se importar com distância)	DFS ou BFS
Explorar todas as possibilidades / backtracking	DFS
Ordenação topológica	DFS (uma das formas)

13. Comparações

Ver seção 6 do 2.B (BFS vs DFS) para a tabela completa comparativa.

14. Casos onde NÃO usar

- Quando o problema pede **menor caminho**/menor número de passos sem pesos — use BFS, não DFS

- Grafos muito profundos com recursão sem cuidado de estouro de pilha

15. Problemas da OBI

- Problemas de "contar regiões" em grid (ex: minas, ilhas, campos) — comuns em Programação Júnior/N1 e N2

16. Outros problemas

- CSES: "Counting Rooms" (flood fill clássico)
- LeetCode: "Number of Islands" (fácil/médio)

17. Cheatsheet

```
cpp

// Grafo (lista de adjacência)
void dfs(int u) {
    visited[u] = true;
    for (int v : adj[u]) if (!visited[v]) dfs(v);
}

// Grid
int dx[] = {-1,1,0,0}, dy[] = {0,0,-1,1};
void dfs(int i, int j) {
    visited[i][j] = true;
    for (int d = 0; d < 4; d++) {
        int ni = i+dx[d], nj = j+dy[d];
        if (ni<0||ni>=n||nj<0||nj>=m) continue;
        if (grid[ni][nj]==1 || visited[ni][nj]) continue;
        dfs(ni, nj);
    }
}
```

18. STL — variação com 8 direções (incluindo diagonais)

```
cpp

int dx8[] = {-1,-1,-1,0,0,1,1,1};
int dy8[] = {-1,0,1,-1,1,-1,0,1};
// mesmo padrão, só troca o array de direções e o loop vai até 8
```

19. Template — contar componentes

```
cpp
```

```
int componentes = 0;
for (int i = 0; i < n; i++) {
    if (!visited[i]) {
        dfs(i);
        componentes++;
    }
}
```

20. Desafios

1. Conte quantas ilhas existem num grid de terra/água.
2. Detecte se um grafo direcionado tem ciclo.
3. Ache o tamanho da maior componente conexa.
4. Pinte (flood fill) uma região a partir de um pixel, como na ferramenta "balde de tinta".
5. Implemente DFS iterativo e compare a ordem de visita com a versão recursiva.

21. Mini-projeto

Implemente uma ferramenta de "balde de tinta" simples: dado um grid de cores e um pixel clicado, repinte toda a região conectada da mesma cor.

22. Revisão

- ☒ Por que DFS não garante menor caminho?
- ☒ Qual é o risco de usar DFS recursivo em grafos muito grandes?
- ☒ Como o padrão $\begin{pmatrix} dx \\ dy \end{pmatrix}$ se aplica tanto a DFS quanto a BFS?

23. Flashcards

Q: DFS serve para achar menor caminho? **A:** Não — DFS só garante alcançabilidade, não a menor distância. Para isso, use BFS (sem peso) ou Dijkstra (com peso).

Q: Quando prefiro DFS iterativo ao recursivo? **A:** Quando o grafo é muito profundo e há risco de estouro de pilha.

24. Resumo final

DFS explora "fundo antes de largo", usando recursão (pilha implícita) ou pilha explícita. Serve para conectividade, componentes, ciclos e flood fill — não para menor caminho. Complexidade $O(V+E)$ em grafos, $O(n \cdot m)$ em grids. Sempre marque visitado antes de continuar.

25. Checklist de maestria

Conhecimento

- ☐ Sei explicar por que DFS não acha menor caminho
- ☐ Sei quando DFS recursivo pode estourar a pilha

Implementação

- ☐ Consigo implementar DFS recursivo em grafo e em grid
- ☐ Consigo implementar DFS iterativo com stack explícita

Reconhecimento

- ☐ Reconheço "contar ilhas/regiões" como flood fill/DFS
- ☐ Reconheço "detectar ciclo" como problema de DFS

Competição

- ☐ Resolvi pelo menos 3 problemas de DFS/flood fill em grid
-

2.B — BFS (Busca em Largura)

1. Introdução

Objetivo: dominar BFS simples, aplicação em menor caminho, em grid, e multi-source BFS — a ferramenta mais importante para o seu ponto fraco atual.

Pré-requisitos: fila (Módulo 1.B), representação de grafos.

Quando estudar: logo depois de filas, antes de Dijkstra.

Nível: ★ Fácil (BFS em grafo simples) → ★★ Médio (BFS em grid, multi-source)

2. Intuição

Imagine jogar uma pedra num lago: as ondas se espalham em círculos concêntricos, alcançando primeiro o que está mais perto. BFS faz exatamente isso — explora "camada por camada", garantindo que a primeira vez que alcança um nó é pela distância mínima (em número de arestas/passos).

3. Problema que motivou

Você tem um mapa (grid) com paredes e espaços livres. Quer saber o menor número de movimentos entre um ponto A e um ponto B, onde cada movimento (cima/baixo/esquerda/direita) custa o mesmo. Um GPS que sempre te dá a rota mais curta usa essa mesma ideia (em grafos com pesos, seria Dijkstra — ver 2.D).

4. Construindo a solução

Solução 1 (força bruta) — DFS testando todos os caminhos: funciona, mas não garante achar o menor caminho de forma eficiente, e pode reprocessar o mesmo nó várias vezes.

Solução final: BFS com fila, processando por camadas, marcando `dist` no momento em que o nó entra na fila $\rightarrow O(V + E)$, ou $O(n \cdot m)$ em grid.

5. Estrutura do algoritmo

```
bfs(origem):
    dist[origem] = 0
    fila.push(origem)
    enquanto fila não vazia:
        u = fila.pop_front()
        para cada vizinho v de u:
            se dist[v] não definida:
                dist[v] = dist[u] + 1
                fila.push(v)
```

6. Implementação — grafo (lista de adjacência)

```
cpp

vector<int> dist(n + 1, -1);
queue<int> q;

dist[origem] = 0;
q.push(origem);

while (!q.empty()) {
    int u = q.front(); q.pop();
    for (int v : adj[u]) {
        if (dist[v] == -1) {
            dist[v] = dist[u] + 1;
            q.push(v);
        }
    }
}
```

7. Implementação — grid (o template que você deve decorar)

```
cpp
```

```

int n, m;
vector<vector<int>> grid; // 0 = livre, 1 = parede
vector<vector<int>> dist(n, vector<int>(m, -1));
queue<pair<int,int>> q;

int dx[] = {-1, 1, 0, 0};
int dy[] = {0, 0, -1, 1};

dist[si][sj] = 0;
q.push({si, sj});

while (!q.empty()) {
    auto [i, j] = q.front(); q.pop();
    for (int dir = 0; dir < 4; dir++) {
        int ni = i + dx[dir], nj = j + dy[dir];
        if (ni < 0 || ni >= n || nj < 0 || nj >= m) continue; // limites
        if (grid[ni][nj] == 1) continue; // obstáculo
        if (dist[ni][nj] != -1) continue; // já visitado
        dist[ni][nj] = dist[i][j] + 1;
        q.push({ni, nj});
    }
}

```

Decore a ordem das 4 checagens: limites → obstáculo → visitado → marcar e empilhar.

8. Complexidade

Operação	Complexidade
BFS completo (grafo)	$O(V + E)$
BFS completo (grid $n \times m$)	$O(n \cdot m)$

9. Visualizações

Grid (S=início, .=livre, #=parede):

```

S . .      Camada 0: S
. # .      Camada 1: vizinhos diretos de S
. . .      Camada 2: vizinhos dos vizinhos
... e assim por diante, em ondas

```

10. Erros clássicos

- Marcar como visitado **depois** de processar em vez de **ao inserir na fila** — o mesmo nó pode entrar várias vezes na fila
- Esquecer de checar limites antes de acessar `grid[ni][nj]`
- Confundir BFS com DFS quando o problema pede explicitamente "menor caminho"
- Esquecer de tratar múltiplas origens (ver seção 18)

11. Problemas clássicos

1. Menor caminho em grafo simples (fácil)
2. Labirinto — menor caminho em grid (médio)
3. Multi-source BFS — "tempo até o fogo alcançar cada célula" (médio/difícil)

12. Padrões — quando pensar em BFS

Se o enunciado diz...	Pense em...
Menor número de movimentos/passos	BFS
Labirinto sem pesos	BFS
Grade (grid), sem peso	BFS
Distância mínima em grafo não ponderado	BFS
Várias fontes/origens simultâneas	Multi-source BFS

13. Comparações — DFS vs BFS vs Dijkstra

Critério	DFS	BFS	Dijkstra
Garante menor caminho?	Não	Sim (sem peso)	Sim (com peso)
Estrutura usada	Pilha (ou recursão)	Fila	Priority Queue
Complexidade	$O(V+E)$	$O(V+E)$	$O((V+E) \log V)$
Uso típico	Conectividade, ciclos	Menor caminho sem peso	Menor caminho com peso

14. Casos onde NÃO usar

- BFS quando as arestas têm pesos diferentes — BFS assume que toda aresta "custa 1"; nesse caso, use Dijkstra
- BFS quando só quer saber "existe caminho?" sem se importar com a distância (DFS é mais simples nesse caso, embora BFS também funcione)

15. Problemas da OBI

- "Labirinto" — menor caminho em grid (frequente em Programação Júnior/N1)
- Problemas de propagação (fogo, água, infecção) em grid — multi-source BFS

16. Outros problemas

- CSES: "Labyrinth", "Message Route" (BFS com reconstrução de caminho)
- LeetCode: "Rotting Oranges" (multi-source BFS clássico)

17. Cheatsheet

```
cpp

queue<pair<int,int>> q;
int dx[]={-1,1,0,0}, dy[]={0,0,-1,1};
dist[si][sj] = 0; q.push({si,sj});
while(!q.empty()){
    auto [i,j]=q.front(); q.pop();
    for(int d=0; d<4; d++){
        int ni=i+dx[d], nj=j+dy[d];
        if(ni<0||ni>=n||nj<0||nj>=m) continue;
        if(grid[ni][nj]==1 || dist[ni][nj]!=-1) continue;
        dist[ni][nj]=dist[i][j]+1;
        q.push({ni,nj});
    }
}
```

18. STL — Multi-source BFS

Quando há múltiplas origens simultâneas, coloque **todas** na fila antes do loop principal, todas com distância 0:

```
cpp
```

```

queue<pair<int,int>> q;
for (auto [i, j] : origens) {
    dist[i][j] = 0;
    q.push({i, j});
}
while (!q.empty()) {
    // resto do BFS é idêntico ao template padrão
}

```

Reconstrução de caminho (guardando de onde veio)

```

cpp

vector<vector<pair<int,int>>> parent(n, vector<pair<int,int>>(m, {-1,-1}));

// dentro do BFS, ao marcar dist[ni][nj]:
parent[ni][nj] = {i, j};

// depois, para reconstruir o caminho do destino até a origem:
vector<pair<int,int>> caminho;
pair<int,int> atual = {destino_i, destino_j};
while (atual != make_pair(-1,-1)) {
    caminho.push_back(atual);
    atual = parent[atual.first][atual.second];
}
reverse(caminho.begin(), caminho.end());

```

19. Template completo (labirinto com reconstrução)

```

cpp

```

```

int n, m, si, sj, ti, tj;
vector<vector<int>> grid;
vector<vector<int>> dist;
vector<vector<pair<int,int>> parent;
int dx[] = {-1,1,0,0}, dy[] = {0,0,-1,1};

int bfs() {
    dist.assign(n, vector<int>(m, -1));
    parent.assign(n, vector<pair<int,int>>(m, {-1,-1}));
    queue<pair<int,int>> q;
    dist[si][sj] = 0;
    q.push({si, sj});
    while (!q.empty()) {
        auto [i, j] = q.front(); q.pop();
        for (int d = 0; d < 4; d++) {
            int ni = i+dx[d], nj = j+dy[d];
            if (ni<0||ni>=n||nj<0||nj>=m) continue;
            if (grid[ni][nj]==1 || dist[ni][nj]!=-1) continue;
            dist[ni][nj] = dist[i][j]+1;
            parent[ni][nj] = {i,j};
            q.push({ni,nj});
        }
    }
    return dist[ti][tj]; // -1 se inalcançável
}

```

20. Desafios

1. Menor caminho num labirinto simples (sem reconstrução).
2. Menor caminho com reconstrução do caminho percorrido.
3. Multi-source BFS: tempo até uma infecção alcançar cada célula.
4. BFS num grafo (não grid) para achar distância mínima entre dois nós.
5. Grid com "portais" (teletransporte) — adapte o BFS para considerar arestas extras.

21. Mini-projeto

Simule um jogo de labirinto onde o jogador pergunta "qual o menor caminho até a saída?" e o programa desenha o caminho encontrado sobre o grid original.

22. Revisão

- ☒ Por que BFS garante menor caminho e DFS não?
- ☒ Qual é a ordem correta das checagens no template de grid?
- ☒ Como adaptar BFS para múltiplas origens?

23. Flashcards

Q: Por que BFS usa fila e não pilha? **A:** Fila garante processamento por camadas (FIFO), o que é essencial para explorar por ordem crescente de distância.

Q: Qual a ordem das checagens no BFS de grid? **A:** Limites → obstáculo → visitado → marcar distância e empilhar.

24. Resumo final

BFS explora por camadas usando fila, garantindo menor caminho em grafos/grids sem peso. Template de grid: 4 checagens em ordem fixa (limites, obstáculo, visitado, processar). Multi-source BFS: todas as origens entram na fila antes do loop, com distância 0.

25. Checklist de maestria

Conhecimento

- ☐ Sei explicar por que BFS garante menor caminho
- ☐ Sei quando usar multi-source BFS

Implementação

- ☐ Consigo escrever o template de BFS em grid sem consultar material
- ☐ Consigo implementar reconstrução de caminho

Reconhecimento

- ☐ Reconheço "menor número de movimentos" como BFS
- ☐ Reconheço "várias origens simultâneas" como multi-source BFS

Competição

- ☐ Resolvi pelo menos 1 problema da OBI usando BFS em grid
 - ☐ Consigo resolver um problema de BFS em grid em menos de 30 minutos
-

2.C — 0-1 BFS

1. Introdução

Objetivo: resolver problemas de menor caminho quando as arestas têm peso **0 ou 1**, sem precisar da complexidade extra do Dijkstra.

Pré-requisitos: BFS (2.B), deque (Módulo 1.B).

Quando estudar: depois de BFS e antes de Dijkstra — é o "meio-termo" entre os dois.

Nível: ★ ★ Médio

2. Intuição

Se todas as arestas pesam 0 ou 1, você ainda quer processar nós em ordem de distância crescente, mas uma fila comum não garante isso quando há arestas de peso 0 misturadas com peso 1. A solução: use um **deque**. Ao relaxar uma aresta de peso 0, insira o vizinho na **frente** do deque (ele "empata" com o nó atual); ao relaxar uma aresta de peso 1, insira no **fim** (ele fica uma "camada" atrás).

3. Problema que motivou

Um grid onde andar em espaço livre custa 0, mas atravessar uma "porta"/obstáculo especial custa 1 (por exemplo, você pode "quebrar" no máximo K paredes). Dijkstra resolveria, mas é mais lento ($O(E \log V)$) do que necessário; 0-1 BFS resolve em $O(V + E)$.

4. Construindo a solução

Solução 1 (Dijkstra): funciona sempre, mas usa `priority_queue` ($O(\log V)$ por operação) — desnecessário quando os pesos são só 0/1.

Solução final (0-1 BFS): usa deque, $O(1)$ por operação $\rightarrow O(V + E)$ total.

5. Estrutura do algoritmo

```
0-1 bfs(origem):
    dist[origem] = 0
    deque.push_front(origem)
    enquanto deque não vazio:
        u = deque.pop_front()
        para cada aresta (u, v, peso):
            se dist[u] + peso < dist[v]:
                dist[v] = dist[u] + peso
                se peso == 0: deque.push_front(v)
                senão: deque.push_back(v)
```

6. Implementação manual (conceito)

A lógica é a mesma do BFS, trocando `queue` por `deque` e decidindo `push_front`/`push_back` conforme o peso da aresta.

7. Implementação STL

```
cpp
```

```

#include <deque>
vector<int> dist(n + 1, INT_MAX);
deque<int> dq;

dist[origem] = 0;
dq.push_front(origem);

while (!dq.empty()) {
    int u = dq.front(); dq.pop_front();
    for (auto [v, peso] : adj[u]) { // adj[u] = {(vizinho, peso 0 ou 1)}
        if (dist[u] + peso < dist[v]) {
            dist[v] = dist[u] + peso;
            if (peso == 0) dq.push_front(v);
            else dq.push_back(v);
        }
    }
}

```

8. Complexidade

Operação	Complexidade
0-1 BFS completo	$O(V + E)$

9. Visualizações

Deque, processando peso 0 → entra na frente:

[v0] [u] [x] [y]

↑ empata com u

Processando peso 1 → entra atrás:

[u] [x] [y] [v1]

↑ fica para depois

10. Erros clássicos

- Usar `queue` comum em vez de `deque` (perde a garantia de ordem)
- Inverter a lógica (colocar peso 1 na frente e peso 0 atrás)
- Esquecer de permitir múltiplas entradas do mesmo nó no deque (assim como no Dijkstra, pode ser necessário relaxar mais de uma vez)
- Usar 0-1 BFS quando os pesos não são só 0 e 1 (nesse caso, precisa de Dijkstra)

11. Problemas clássicos

1. Grid onde mover custa 0, mas "quebrar parede" custa 1 (médio)
2. Problemas de "número mínimo de trocas" onde a troca custa 1 e mover livre custa 0 (médio/difícil)

12. Padrões — quando pensar em 0-1 BFS

Se o enunciado diz...	Pense em...
Pesos são só 0 e 1	0-1 BFS
"Quebrar no máximo K obstáculos" com custo 1 por quebra	0-1 BFS ou Dijkstra

13. Comparações

Ver tabela na seção 13 do 2.B (DFS vs BFS vs Dijkstra). 0-1 BFS fica entre BFS e Dijkstra: mais rápido que Dijkstra, mas só funciona com pesos 0/1.

14. Casos onde NÃO usar

- Quando os pesos variam além de 0 e 1 — use Dijkstra
- Quando não há pesos nenhum — BFS simples já resolve, sem necessidade de deque

15. Problemas da OBI

- Menos comuns nas fases iniciais da OBI, mas podem aparecer em variações de "labirinto com custo de quebrar parede"

16. Outros problemas

- CSES: problemas de grid com custo variável
- Codeforces: busca por tag "0-1 bfs"

17. Cheatsheet

```
cpp
```

```

deque<int> dq;
dist[origem] = 0;
dq.push_front(origem);
while (!dq.empty()) {
    int u = dq.front(); dq.pop_front();
    for (auto [v, peso] : adj[u]) {
        if (dist[u] + peso < dist[v]) {
            dist[v] = dist[u] + peso;
            peso == 0 ? dq.push_front(v) : dq.push_back(v);
        }
    }
}

```

18. STL — versão em grid

```

cpp

// mesmo template de BFS em grid, mas com deque<pair<int,int>>
// e decidindo push_front/push_back conforme o "custo" de entrar na célula vizi

```

19. Template

Ver seção 7 (implementação STL) — é o template completo e pronto para adaptar.

20. Desafios

1. Grid onde é permitido "quebrar" até K paredes; ache o menor custo até o destino.
2. Adapte o BFS de labirinto do módulo 2.B para aceitar pesos 0/1.
3. Compare o tempo de execução de 0-1 BFS vs Dijkstra no mesmo problema.

21. Mini-projeto

Simule um jogo onde mover para célula livre é grátis, mas "abrir uma porta trancada" custa 1 unidade de energia (limitada); ache o caminho de menor custo de energia.

22. Revisão

- ✓ Por que 0-1 BFS usa deque em vez de queue simples?
- ✓ Qual a diferença de complexidade entre 0-1 BFS e Dijkstra?

23. Flashcards

Q: Quando uma aresta de peso 0 é processada, ela vai para onde no deque? **A:** Para a frente (push_front).

Q: 0-1 BFS é mais rápido que Dijkstra. Por quê? **A:** Porque usa deque $O(1)$ por operação, em vez de `priority_queue` $O(\log V)$.

24. Resumo final

0-1 BFS é o meio-termo entre BFS e Dijkstra: usa deque, coloca peso 0 na frente e peso 1 atrás, mantendo processamento por ordem de distância crescente em $O(V+E)$. Só funciona quando todos os pesos são 0 ou 1.

25. Checklist de maestria

Conhecimento

- ☐ Sei explicar por que 0-1 BFS funciona com deque
- ☐ Sei quando 0-1 BFS não se aplica (pesos fora de $\{0,1\}$)

Implementação

- ☐ Consigo implementar 0-1 BFS do zero

Reconhecimento

- ☐ Reconheço "pesos 0 e 1" como sinal de 0-1 BFS

Competição

- ☐ Resolvi pelo menos 1 problema de 0-1 BFS
-

2.D — Dijkstra

1. Introdução

Objetivo: resolver menor caminho em grafos com pesos positivos quaisquer, incluindo reconstrução de caminho.

Pré-requisitos: `priority_queue` (Módulo 1.B), BFS (2.B).

Quando estudar: depois de BFS e 0-1 BFS.

Nível: ★ ★ ★ Difícil (conceitualmente simples, mas com detalhes de implementação delicados)

2. Intuição

Imagine um GPS calculando a rota mais rápida entre duas cidades, onde cada estrada tem um tempo de viagem diferente. Dijkstra sempre expande primeiro a cidade "mais barata de alcançar até agora" — como um explorador que sempre escolhe o caminho mais curto conhecido até o momento, atualizando suas estimativas conforme descobre atalhos.

3. Problema que motivou

Você tem uma rede de cidades conectadas por estradas com pesos (distância/tempo/custo) diferentes. Quer saber o menor custo total para ir de uma cidade A até todas as outras (ou até uma cidade B específica).

4. Construindo a solução

Solução 1 (BFS ingênuo): ignora os pesos, trata toda aresta como custo 1 → resposta errada quando os pesos diferem.

Solução 2 (Bellman-Ford): funciona até com pesos negativos, mas é $O(V \cdot E)$, mais lento.

Solução final (Dijkstra): usa `priority_queue` para sempre processar o nó de menor distância acumulada primeiro → $O((V+E) \log V)$. **Só funciona com pesos não-negativos.**

5. Estrutura do algoritmo

```
dijkstra(origem):
    dist[todos] = infinito
    dist[origem] = 0
    pq.push((0, origem))
    enquanto pq não vazia:
        (d, u) = pq.pop() // menor distância primeiro
        se d > dist[u]: continue // entrada desatualizada
        para cada aresta (u, v, peso):
            se dist[u] + peso < dist[v]:
                dist[v] = dist[u] + peso
                pq.push((dist[v], v))
```

6. Implementação manual (conceito de relaxamento)

"Relaxar" uma aresta (u, v, peso) significa: se ir até u e depois para v for mais barato do que a melhor distância conhecida até v , atualize $\text{dist}[v]$.

```
cpp

bool relax(int u, int v, int peso, vector<long long>& dist) {
    if (dist[u] + peso < dist[v]) {
        dist[v] = dist[u] + peso;
        return true; // houve melhoria
    }
    return false;
}
```

7. Implementação STL completa

```
cpp

#include <queue>
const long long INF = 1e18;

vector<vector<pair<int,int>>> adj(n + 1); // adj[u] = {(vizinho, peso)}
vector<long long> dist(n + 1, INF);
vector<int> parent(n + 1, -1);

priority_queue<pair<long long,int>, vector<pair<long long,int>>, greater<>> pq;

dist[origem] = 0;
pq.push({0, origem});

while (!pq.empty()) {
    auto [d, u] = pq.top(); pq.pop();
    if (d > dist[u]) continue; // entrada desatualizada, ignore

    for (auto [v, peso] : adj[u]) {
        if (dist[u] + peso < dist[v]) {
            dist[v] = dist[u] + peso;
            parent[v] = u;
            pq.push({dist[v], v});
        }
    }
}

// Reconstrução do caminho até um destino:
vector<int> caminho;
for (int at = destino; at != -1; at = parent[at]) caminho.push_back(at);
reverse(caminho.begin(), caminho.end());
```

8. Complexidade

Operação	Complexidade
Dijkstra completo	$O((V + E) \log V)$

9. Visualizações

```
Grafo com pesos:
  (2)      (5)
A ----- B ----- C
```

```
\
  \___(10)___/

dist[A]=0
dist[B]=2 (via A)
dist[C]=7 (via A→B→C, melhor que 10 direto)
```

10. Erros clássicos

- Usar `int` em vez de `long long` para distâncias (overflow em grafos grandes)
- Esquecer o `if (d > dist[u]) continue;` (processar entradas desatualizadas gera trabalho extra, embora normalmente não gere resposta errada, é um desperdício grande de tempo)
- Usar Dijkstra com **pesos negativos** — não funciona corretamente; nesse caso use Bellman-Ford
- Esquecer de inicializar `dist[]` com infinito
- Esquecer o min-heap (`greater<>`) — usar o `priority_queue` padrão (max-heap) inverte a lógica

11. Problemas clássicos

1. Menor caminho de A até todos os outros nós (fácil/médio)
2. Menor caminho com reconstrução de rota (médio)
3. Menor caminho com múltiplas restrições (ex: menor caminho entre os K mais baratos) (difícil)

12. Padrões — quando pensar em Dijkstra

Se o enunciado diz...	Pense em...
Custo/peso/distância variável nas arestas	Dijkstra
"Caminho mínimo" com pesos positivos	Dijkstra
Pesos podem ser negativos	Bellman-Ford (não Dijkstra)

13. Comparações

Ver tabela na seção 13 do 2.B. Resumo: BFS (sem peso) < 0-1 BFS (pesos 0/1) < Dijkstra (pesos positivos quaisquer) < Bellman-Ford (aceita pesos negativos, mas mais lento).

- Grafos com arestas de peso negativo (use Bellman-Ford)
- Quando todas as arestas têm o mesmo peso (BFS simples já resolve, mais rápido e simples)

15. Problemas da OBI

- Problemas de "menor custo de viagem entre cidades" com pesos nas estradas — típico de fases N2/Sênior

16. Outros problemas

- CSES: "Shortest Routes I" e "Shortest Routes II"
- Codeforces: buscar por tag "shortest-paths"

17. Cheatsheet

```
cpp

priority_queue<pair<long long,int>, vector<pair<long long,int>>, greater<>> pq;
dist[origem] = 0;
pq.push({0, origem});
while (!pq.empty()) {
    auto [d, u] = pq.top(); pq.pop();
    if (d > dist[u]) continue;
    for (auto [v, peso] : adj[u])
        if (dist[u] + peso < dist[v]) {
            dist[v] = dist[u] + peso;
            pq.push({dist[v], v});
        }
}
```

18. STL — Dijkstra em grid (custo de entrar em cada célula)

```
cpp
```

```

priority_queue<tuple<long long,int,int>, vector<tuple<long long,int,int>>, greater<tuple<long long,int,int>>>,
dist[si][sj] = 0;
pq.push({0, si, sj});
while (!pq.empty()) {
    auto [d, i, j] = pq.top(); pq.pop();
    if (d > dist[i][j]) continue;
    for (int dir = 0; dir < 4; dir++) {
        int ni = i+dx[dir], nj = j+dy[dir];
        if (ni<0||ni>=n||nj<0||nj>=m) continue;
        if (grid[ni][nj]==1) continue; // parede, se houver
        long long custo = d + custoEntrar(ni, nj); // custo da célula destino
        if (custo < dist[ni][nj]) {
            dist[ni][nj] = custo;
            pq.push({custo, ni, nj});
        }
    }
}
}

```

19. Template

Ver seção 7 — implementação STL completa, já pronta para copiar e adaptar em prova.

20. Desafios

1. Implemente Dijkstra e reconstrua o caminho de menor custo.
2. Adapte Dijkstra para funcionar num grid com custo de entrada por célula.
3. Compare o resultado de BFS ingênuo vs Dijkstra num grafo com pesos diferentes — mostre onde BFS erra.
4. Implemente Dijkstra para achar o menor caminho entre dois nós específicos (parando cedo quando o destino é retirado da priority_queue).
5. Resolva um problema com múltiplas origens em Dijkstra (mesma ideia do multi-source BFS).

21. Mini-projeto

Simule um sistema de rotas de entrega: cidades conectadas por estradas com tempos diferentes; para um pedido, calcule a rota mais rápida do depósito até o cliente e mostre o caminho percorrido.

22. Revisão

- ✓ Por que Dijkstra não funciona com pesos negativos?
- ✓ Por que é necessário o `if (d > dist[u]) continue;`?
- ✓ Por que `long long` é importante nas distâncias?

23. Flashcards

Q: Qual estrutura de dados Dijkstra usa para sempre processar o nó mais barato primeiro?

A: Priority queue (min-heap).

Q: Dijkstra funciona com pesos negativos? **A:** Não — nesse caso, use Bellman-Ford.

24. Resumo final

Dijkstra encontra menor caminho em grafos com pesos não-negativos, usando `priority_queue` para sempre expandir o nó de menor distância acumulada. Complexidade $O((V+E) \log V)$. Cuidados: `long long`, min-heap, checar entradas desatualizadas, nunca usar com pesos negativos.

25. Checklist de maestria

Conhecimento

- ☐ Sei explicar por que Dijkstra falha com pesos negativos
- ☐ Sei explicar o papel do "relaxamento" de arestas

Implementação

- ☐ Consigo implementar Dijkstra do zero, incluindo reconstrução de caminho
- ☐ Consigo adaptar Dijkstra para grid

Reconhecimento

- ☐ Reconheço "pesos variados/custos diferentes" como sinal de Dijkstra
- ☐ Sei diferenciar quando usar BFS, 0-1 BFS ou Dijkstra

Competição

- ☐ Resolvi pelo menos 2 problemas com Dijkstra
- ☐ Consigo resolver um problema de Dijkstra em menos de 30 minutos

Mapa Mental de Reconhecimento — Módulo 2 (referência rápida)

Se o enunciado diz...	Pense em...
Menor número de movimentos	BFS
Labirinto sem pesos	BFS
Grade (grid)	BFS/DFS
Distância mínima, sem peso	BFS

Se o enunciado diz...	Pense em...
Pesos 0 e 1	0-1 BFS
Pesos positivos quaisquer	Dijkstra
Pesos podem ser negativos	Bellman-Ford
Quer visitar tudo, sem menor caminho	DFS
Contar regiões/ilhas	DFS ou BFS (flood fill)
Detectar ciclo	DFS